به نام خدا

ساختمان داده‌ها و الگوریتم‌ها (۴۰۲۵۴)

دانشگاه صنعتی شریف — تمرین سوم

مدرس: مهدی صفرنژاد — انتشار: ۶ آبان ۹۸

درخت‌ها

زمان تحویل این تمرین (شامل بخش نظری و بخش عملی) شنبه ۱۷ آبان ۹۸ ساعت ۱:۰۰ بامداد (یک ساعت پس از نیمه‌شب جمعه) می‌باشد. به توضیحات سایت درس دربارهٔ تمرین‌ها، تحویل آن و تأخیر توجه کنید. تمرین‌ها از سایت کوئرا تحویل گرفته می‌شوند. قسمت‌های نظری به صورت فایل PDF تحویل گرفته می‌شوند که این فایل یا باید با حروف‌چینی رایانه‌ای تولید شده باشد و یا اگر دست‌نوشته است، توسط نرم‌افزارهایی مثل CamScanner یا Office Lens و یا با دستگاه اسکنر تصویربرداری شده باشد.

سؤال‌های ۱ تا ۴ تئوری و سؤال‌های ۵ تا ۷ عملی هستند. تمامی بخش‌ها را از طریق کوئرا ارسال کنید.

سؤال ۱. [۸ نمره] مقادیر ۵، ۱۰، ۸، ۷، ۱۳، ۱۲، ۴، ۶ را به ترتیب از راست به چپ در درخت های خالی Red Black و AVL اضافه کنید و دو درخت حاصل را رسم کنید. سپس مقادیر های ۶ و ۸ و ۱۲ را به ترتیب از راست به چپ، از دو درخت حذف کنید و دوباره حاصل را رسم کنید. (می‌توانید برای مرور مفاهیم از این‌جا کمک بگیرید!)

پاسخ:

سؤال ۲. [۱۰ نمره]

مسئلهٔ مرتب‌سازی اعداد را در نظر بگیرید. فرض کنید اعداد به صورت دقیق مشخص نیستند. در عوض برای هر عدد، یک بازهٔ بسته روی اعداد حقیقی داریم که می‌دانیم آن عدد به آن بازه متعلق است. یعنی n بازه به صورت $[a_i, b_i]$ داریم، که در آن $a_i \leq b_i$.

هدف این است که این بازه‌ها را مرتب کنیم؛ یعنی یک جایگشت $\langle i_1, i_2, ..., i_n \rangle$ از بازه‌ها بسازیم به طوری که برای هر $j = 1, 2, ..., n$ مقادیر $c_i \in [a_{i_j}, b_{i_j}]$ موجود باشند که در نامساوی $c_1 \leq c_2 \leq ... \leq c_n$ صدق کنند.

فرض کنید که هر بازه، حداقل با $d - 1$ بازه‌ی دیگر همپوشانی داشته باشد. الگوریتمی از $O(n \log \frac{n}{d})$ ارائه دهید که این n بازه با درجه‌ی d را مرتب کند. بدین ترتیب، اگر $d = \Theta(1)$ باشد، الگوریتم در زمان $O(n \log n)$ اجرا می‌شود، و اگر $d = \Theta(n)$ باشد، الگوریتم در زمان $O(n)$ اجرا خواهد شد.

پاسخ: به این نوع مرتب‌سازی بازه‌ها، مرتب سازی فازی (Fuzzy Sort) نیز می‌گویند. الگوریتمی با رویکرد تقسیم و حل و مشابه مرتب‌سازی سریع ارائه می‌دهیم. در هر مرحله از اجرای الگوریتم Fuzzy Sort، یکی از بازه‌ها را به عنوان محور (Pivot) قرار می‌دهیم و بازه‌ها را به دو گروه تقسیم می‌کنیم: گروه چپ و گروه راست. الگوریتم Fuzzy Sort را به صورت بازگشتی روی این دو دسته صدا می‌زنیم، تا زمانی که تعداد بازه‌های دسته حداکثر d باشد، که در این حالت، بازه‌ها را در زمان $O(d)$ مرتب می‌کنیم.

ناوردای این الگوریتم به این صورت است که در هر مرحله از اجرای الگوریتم روی مجموعه‌ای از بازه‌ها (S)، این مجموعه می‌تواند به سه دسته‌ی چپ، وسط، و راست تقسیم کرد، به طوری که شرایط زیر برقرار باشد:

۱. دسته‌ی چپ، دسته‌ی کوچک‌ترین بازه‌هاست. به بیان دیگر، هر بازه در دسته‌ی چپ، می‌تواند در ترتیب خروجی، قبل از هر بازه‌ای در دسته‌های وسط و راست قرار بگیرد.

۲. دسته‌ی راست، دسته‌ی بزرگ‌ترین بازه‌هاست. به بیان دیگر، هر بازه در دسته‌ی راست، می‌تواند در ترتیب خروجی، بعد از هر بازه‌ای در دسته‌های وسط و چپ قرار بگیرد.

۳. در دسته‌ی چپ، هر دو بازه با یک‌دیگر همپوشانی دارند.

۴. در دسته‌ی وسط، هر بازه با حداقل $d-1$ بازه‌ی دیگر (نه لزوماً در دسته‌ی وسط) همپوشانی دارد.

۵. در دسته‌ی راست، هر دو بازه با یک‌دیگر همپوشانی دارند.

شرح الگوریتم:

همان طور که اشاره شد، این الگوریتم بر پایه‌ی مرتب‌سازی سریع طراحی می‌شود. ابتدا نقاط ابتدایی بازه‌ها (نقاط سمت چپ) را در نظر گرفته و میانه‌ی آن‌ها را در زمان $O(n)$ محاسبه می‌کنیم. بازه‌ی متناظر این نقطه را به عنوان محور در نظر می‌گیریم و بازه‌ها را بر اساس نقاط ابتدایی‌شان partition می‌کنیم (زیرا می‌خواهیم زمان اجرای الگوریتم در بدترین حالت نیز $O(n \log \frac{n}{d})$ باشد). این عمل، مسئله را به دو زیرمسئله تقسیم می‌کند: زیرمسئله‌ی سمت چپ و سمت راست، که اندازه‌ی هر کدام $\lfloor \frac{n}{2} \rfloor$ است.

از آن‌جایی که همه‌ی بازه‌های زیرمسئله‌ی چپ، نقاط ابتدایی کوچک‌تری نسبت به محور دارند، همه‌ی آن‌ها می‌توانند در خروجی قبل از محور قرار گیرند. به طور مشابه، تمام بازه‌های زیرمسئله‌ی راست می‌توانند در خروجی بعد از محور قرار گیرند. در نتیجه، Fuzzy Sort به صورت بازگشتی روی زیرمسئله‌های چپ و راست به صورت مستقل صدا زده می‌شود.

حال با استفاده از استقرا، ثابت می‌کنیم که ناوردای الگوریتم در هر مرحله از صدا زدن تابع بازگشتی، ثابت می‌ماند. در آغاز، شرایط ناوردا برقرار هستند؛ زیرا می‌توانیم دسته‌ی چپ و راست را خالی بگذاریم و تمام بازه‌ها را در دسته‌ی وسط قرار دهیم.

در ادامه، فرض کنید که ناوردا، در قسمتی از اجرای الگوریتم، برای مجموعه‌ی S از بازه‌ها برقرار باشد. طبق فرض استقرا، می‌توان S را به سه دسته‌ی S_L، S_M و S_R افراز کرد (که به ترتیب، همان دسته‌های چپ، وسط و راست هستند). برای زیرمسئله سمت چپ محور، سه دسته‌ی ناوردا را مشخص می‌کنیم:

فرض کنید S^l مجموعه‌ی بازه‌های زیرمسئله‌ی چپ باشد. دو حالت پیش می‌آید:

۱. محور در S_L قرار دارد. قرار می‌دهیم: $S^l_L := S^L, S^l_R := \emptyset, S^l_M := \emptyset$. این سه مجموعه، در شرایط ناوردا صدق می‌کنند.

۲. محور در S_L قرار ندارد. پس می‌توان گفت $S_L \subset S^l$. فرض کنید T مجموعه‌ی بازه‌هایی در S^l باشد که با محور همپوشانی دارند. قرار می‌دهیم:

$$S^l_L := S_L, S^l_R := T \setminus S^l_L, S^l_M := S^l \setminus (S^l_L \cup S^l_R)$$

این سه مجموعه، در شرایط ناوردا صدق می‌کنند.

پس ثابت شد که ناوردا برای زیرمسئله‌ی سمت چپ برقرار است. به صورت مشابه، می‌توان برقرار بودن آن را برای زیرمسئله‌ی سمت راست نیز ثابت کرد.

در مرحله‌ی بعد، ادعا می‌کنیم که یک زیرمسئله با تعداد بازه‌های حداکثر d را می‌توان در زمان خطی مرتب کرد. یک مجموعه از بازه‌ها با حداکثر d عضو را در نظر بگیرید. طبق ناوردای الگوریتم، می‌توان بازه‌ها را به سه دسته (که در بالا مشخص شده‌اند) افراز کرد. دو حالت پیش می‌آید. اگر دسته‌ی وسط خالی باشد، یک حالت مرتب‌سازی معتبر این است که که بازه‌های دسته‌ی چپ را به ترتیب دلخواه قرار دهیم، و پس از آن‌ها، بازه‌های دسته‌ی راست را به ترتیب دلخواه قرار دهیم. و اگر دسته‌ی وسط حداقل یک عضو داشته باشد، این بازه‌ها با تمام $d-1$ بازه‌ی دیگر همپوشانی دارند. پس باز هم می‌توان به سادگی بازه‌ها را مرتب کرد: ابتدا بازه‌هایی که با نقطه‌ی ابتدایی یکی از بازه‌های دسته‌ی وسط (I) همپوشانی دارند، سپس بازه‌ی I، و در آخر نیز تمام بازه‌هایی که با نقطه‌ی انتهایی بازه‌ی I همپوشانی دارند را قرار می‌دهیم.

بنابراین ما می‌توانیم درخت بازگشتی را در زمانی که به زیرمسئله‌ای با اندازه‌ی حداکثر d می‌رسیم، هرس کنیم. پس عمق درخت برابر با $\Theta(\lg \frac{n}{d})$ است، و در نتیجه زمان اجرای برنامه $O(n \lg \frac{n}{d})$ است.

سؤال ۳. [۱۰ نمره]

در یک مهمانی n فرد با بارانی‌های مشابه که فقط اندازه‌های متفاوت دارند، وارد می‌شوند و آن‌ها را در یک اتاق قرار می‌دهند. در پایان مهمانی مشخص نیست کدام بارانی برای چه کسی است. می‌خواهیم بارانی هر فرد را مشخص کنیم. تنها کاری که مجاز به انجام آن هستیم، این است که یک بارانی را بر تن یک نفر امتحان کنیم که یا اندازه است، یا کوچک است و یا بزرگ. فرض کنید اندازهٔ بارانی‌ها با هم و همچنین افراد با هم دو به دو متفاوت است و نمی‌توانیم دو بارانی و یا دو نفر را با هم مقایسه کنیم. الگوریتمی از مرتبهٔ زمانی متوسط $O(n \log n)$ پیشنهاد دهید و درستی آن را ثابت کنید.

راهنمایی: می‌توانید از ایده‌ای مشابه مرتب‌سازی سریع تصادفی استفاده کنید.

پاسخ: یکی از افراد را به تصادف به عنوان لولا (محور) در نظر می‌گیریم و تمام بارانی‌ها را با او امتحان می‌کنیم. بنابراین یک بارانی برایش اندازه، تعدادی کوچکتر و تعدادی بزرگتر می‌شوند. بارانی‌ای که اندازه بود را در نظر بگیرید. این بارانی را با همهٔ افراد امتحان می‌کنیم. افراد نیز به سه دستهٔ مشابه افراز می‌شوند. فرد انتخابی و بارانی او را کنار گذاشته و دو دستهٔ بارانی‌ها و افراد کوچکتر و بارانی‌ها و افراد بزرگتر را به صورت بازگشتی حل می‌کنیم. زمان اجرا به صورت تصادفی (میانگین) $O(n \log n)$ خواهد شد (با تحلیلی مشابه الگوریتم مرتب‌سازی سریع تصادفی).

سؤال ۴. [۱۲ نمره]

به پرسش‌های زیر درباره درخت‌های دودویی جست‌وجو پاسخ دهید.

آ. [۶ نمره] رابطه‌ای بازگشتی برای تعداد ددج‌های متفاوتی که می‌توان با اعداد $a_1 < a_2 < ... < a_n$ نوشت به دست آورید. حال فرض کنید یک ددج ثابت داریم. رابطه‌ای بازگشتی برای تعداد دنباله‌های متفاوتی از اعضای این ددج را بیابید که درج آن‌ها منجر به ساخت این ددج شود.

ب. [۶ نمره] فرض کنید برای ساخت یک دنباله از اعضای یک ددج، به ترتیب از بیشترین عمق به کمترین عمق و در هر عمق از چپ به راست اعداد داخل گره‌های ددج را می‌نویسیم. آیا به ازای هر دنباله متناهی از اعداد صحیح متمایز، ددج متناظر با آن وجود دارد؟ درصورتی که مطمئن باشیم حداقل یک ددج متناظر با یک دنباله وجود دارد، آیا این ددج یکتاست؟

پاسخ: آ) برای هر درخت دودویی ریشه‌دار (که جهت چپ و راست آن نیز متمایز است) با n گره تنها یک ترتیب از مقداردهی $a_1, ..., a_n$ وجود دارد که درخت را تبدیل به ددج کند و هر ددج نیز یک درخت دودویی ریشه‌دار است بنابراین کافیست تعداد این درخت‌ها را بشماریم. اگر تعداد گره‌های زیردرخت چپ برابر با i باشد تعداد حالات ساخت درخت برابر است با $T(i)T(n-1-i)$ بنابراین تعداد کل حالات برابر است با

$$T(n) = \sum_{i=0}^{n-1} T(i)T(n-1-i)$$

از آنجایی که عضو اول دنباله درج حتما ریشه است مکان آن در دنباله به طور یکتا مشخص می‌شود. اگر تعداد دنباله‌های درج را با f، تعداد دنباله‌های درج برای زیردرخت‌های چپ و راست را به ترتیب با f_l و f_r و تعداد گره‌های زیردرخت‌های چپ و راست را به ترتیب با n_l و n_r نمایش دهیم آنگاه برای ادامه دنباله درج باید ابتدا مکان n_l گره زیردرخت چپ در دنباله را انتخاب کرده و سپس تعداد حالات مرتب کردن گره‌های زیردرخت چپ و زیردرخت راست را بشمریم بنابراین نهایتا خواهیم داشت

$$f = \binom{n_l + n_r}{n_l} f_l f_r$$

ب) به ازای هر دنباله دلخواه چنین ددج‌ای الزاما وجود ندارد. برای مثال دنباله ۲ ,۱ ,۳ را در نظر بگیرید. ۲ حتما باید ریشه باشد. ۱ نیز حتما فرزند ۲ است و چون از ۲ کمتر است فرزند چپ ۲ است. از آنجایی که فرزند چپ ریشه بلافاصله قبل از ریشه آمده پس ریشه فرزند راستی ندارد. در این صورت هیچ مکانی برای قرار دادن ۳ نداریم. بنابراین ددج‌ای متناظر با دنباله فوق وجود ندارد.

در صورتی که از وجود یک ددج برای یک دنباله مطمئن باشیم، می‌توانیم ثابت کنیم چنین ددج‌ای یکتاست. برای اینکار ابتدا نشان می‌دهیم اگر یک ددج برای دنباله n تایی وجود داشته باشد، برای هر پسوند k تایی این دنباله نیز یک ددج وجود دارد. برای اینکار کافیست مرتبا راس‌هایی که در ابتدای لیست می‌آیند را حذف کنیم. حال با کمک استقرا یکتایی این ددج برای همه k ها از جمله n را نشان می‌دهیم. فرض کنید ددج متناظر با پسوند kتایی یکتا باشد. در این صورت اگر از هر ددج متناظر با پسوند $k+1$تایی گره $k+1$اُم را حذف کنیم به یک ددج یکتا می‌رسیم. سپس باید گره $k+1$اُم را به عنوان یک برگ در این ددج یکتا درج کنیم که حاصل یک ددج یکتا می‌شود. بنابراین ددج متناظر با پسوند $k+1$تایی نیز یکتاست. پایه استقرا نیز حالتی است که فقط ریشه

وجود دارد که بدیهتا یکتاست. بنابراین با کمک استقرا ثابت کردیم اگر برای یک دنباله nتایی حداقل یک ددج معادل وجود داشته باشد، این ددج معادل یکتاست.

سؤال ۵. [۲۵ نمره]

در یک خیابان n خانه با ارتفاع های متفاوت وجود دارد. به خانه A می گوییم بر خانه B مشرف است اگر ارتفاع A از ارتفاع B بیشتر باشد. هدف ما در این مسئله این است که برای هر خانه تعداد خانه هایی را که در سمت راستش هستند و بر آن مشرف هستند را بیابیم.

در این سوال **حتما** باید درخت AVL را پیاده‌سازی کنید.

ورودی:
ارتفاع خانه های خیابان از چپ به راست در یک خط آمده است.

خروجی:
به ترتیب از چپ به راست چاپ می کنیم که بر هر خانه چند خانه دیگر در سمت راستش مشرف هستند.

محدودیت‌ها:
$۱ \leq n \leq ۴ \times ۱۰^۴$
که n همان تعداد خانه ها است.

ورودی و خروجی نمونه:

stdin	stdout
0 1 2 3 4	4 3 2 1 0

stdin	stdout
4 2 0 1	0 0 1 0

پاسخ: در این مسئله کافیست شروع کنیم و از راست به چپ ارتفاع ساختمان هارا در یک avl tree بریزیم. در هر مرحله وقتی ارتفاع یک ساختمان را اضافه کردیم کافیست چک کنیم که چند عدد بزرگتر از آن در درخت موجود است. برای این کار علاوه بر اطلاعات درخت برای هر راس تعداد رئوسی که این راس پدرش هست را هم نگه می داریم و در $O(\log n)$ می توانیم بفهمیم چند عدد بزرگتر به درخت اضافه شده است. همین کار را تا انتها انجام می دهیم و مرتبه زمانی الگوریتم هم $O(n \log n)$می شود.

سؤال ۶. [۲۰ نمره]

شایان که به تازگی از جلسات مهمی اخراج شده است، در n کلاس مشغول تدریس المپیاد است. در راستای جمع کردن کلاس‌ها با مساله‌ای روبه‌رو شده، از آنجایی که وقتی برای حل کردن سوال برایش باقی نمانده، از کیوان که در این روزها بسیار بیکار است می‌خواهد که مسأله زیر را برایش حل کند:
در ابتدا کلاس‌های شایان خالی هستند، او q درخواست دارد که به یکی از شکل‌های زیر می‌باشند.

۱. یک دانش‌آموز با نمره x به کلاس c اضافه شود.

۲. نمره تمام دانش‌آموزان کلاس c با x جمع شود.

۳. دانش‌آموزی که کمترین نمره را در بین تمام دانش‌آموزان شایان دارد، اخراج کرده و نمره‌اش را گزارش کنید (اگر دانش‌آموزی وجود نداشت -۱ را گزارش کنید).

از آنجایی که کیوان دیگر توانایی‌های سابق را ندارد، از شما می‌خواهد تا به درخواست‌ها پاسخ دهید تا اندک آبرویی که برایش مانده حفظ شود.
توجه کنید که نمرات شایان می‌تواند هر عدد صحیحی باشد (حتی منفی).

ورودی:
در خط اول ورودی n و q که تعداد کلاس‌ها و تعداد درخواست‌های شایان هستند، داده می‌شود.
در q خط بعدی به ترتیب درخواست‌ها به شکل t_i و اگر $t_i \neq ۳$ در ادامه دو عدد c_i و x_i نیز داده می‌شود که t_i نوع درخواست iاُم می‌باشد.

خروجی:
به ازای هر درخواست نوع ۳ عدد خواسته شده را در یک خط گزارش کنید.
محدودیت‌ها:
$۱ \leq n, q \leq ۱۰^۵$
$۱ \leq t_i \leq ۳$
$۱ \leq c_i \leq n$
$-۱۰^۹ \leq x_i \leq ۱۰^۹$

ورودی و خروجی نمونه:

stdin	stdout
5 11 1 1 5 1 1 3 1 2 2 3 3 3 3 1 1 10 1 2 5 2 2 10 3	2 3 5 -1 10

پاسخ: به ازای هر کلاس یک هرم در نظر می‌گیریم و یک هرم دیگر نیز داریم که کمترین نمره هر کلاس و به همراه شماره کلاس آن در این هرم قرار دارد. برای هر کلاس یک آرایه $toAdd$ نیز داریم که نشان می‌دهد تا الان چقدر درخواست نوع ۲ به نمرات این کلاس اضافه کرده. برای مدیریت درخواست نوع ۱، باید نمره $x - val[c]$ را به کلاس c اضافه کنیم و برای مدیریت درخواست نوع ۳، باید همیشه هرمی که کمترین نمره کلاس‌ها را نگه می‌دارد، به‌روز نگه داریم، که این هم کار سختی نیست. در کل هر درخواست را توانستیم $\Theta(logq)$ پاسخ دهیم.

سؤال ۷. [۱۵ نمره]

یکی از قدرت‌های ماوراء الطبیعه‌ی شنگدباو قدرت میانه‌روی اوست! او به قدری میانه‌رو است که آقاصادق یک روز به او کلی عدد به ترتیب می‌گوید و پس از هر بار شنیدن یک عدد جدید شنگدباو میانه‌ی همه‌ی اعدادی که تا آن‌جا شنیده را به سرعت اعلام می‌کند!

میانه‌ی n عدد برابر عدد $\lceil \frac{n}{۲} \rceil$ام در ترتیب مرتب شده‌شان است. یعنی برای مثال میانه‌ی سه عدد ۱، ۱۰ و ۵ برابر ۵ می‌باشد. یا میانه‌ی چهار عدد ۱، ۱۰، ۱۱، ۱۰۰ برابر ۱۰ می‌باشد.

آیا می‌توانید مثل شنگدباو باشید؟

ورودی:

در ورودی ابتدا به‌شما عدد q داده می‌شود که تعداد اعدادی است که آقاصادق به شنگدباو می‌گوید و سپس در q خط بعدی در هر خط یک عدد x می‌آید که عددی است که آقاصادق در آن مرحله اعلام کرده‌است.

خروجی:

در q خط خروجی دهید و در خط iام میانه‌ی اعداد اول تا iام را اعلام کنید.

محدودیت‌ها:

$$۱ \leq q \leq ۱۰^۵ \ , \ ۱ \leq x \leq ۱۰^۹$$

ورودی و خروجی نمونه

stdin	stdout
5 1 10 3 4 9	1 1 3 3 4

پاسخ: دو تا درخت هرمی استفاده می‌کنیم یکی درخت هرمی کمینه از عناصر بعد از میانه و دیگری درخت هرمی بیشینه از عناصر قبل از میانه‌است (شامل خود میانه). ساختن درخت هرمی بیشینه همانند همان درخت هرمی کمینه‌است فقط هر رأس از بچه‌هایش بزرگتر است و ریشه بیشینه می‌باشد.

در این‌صورت همیشه بزرگترین عنصر هرم بیشینه برابر جواب است. هرگاه عددی اضافه می‌کنیم، آن‌را به هرم بیشینه اضافه می‌کنیم و اگر لازم بود بزرگترین عنصر هرم بیشینه را حذف کرده و به هرم کمینه اضافه می‌کنیم. به این‌صورت همیشه در این دو هرم اعداد نیمه‌ی اول و دوم به صورت مجزا باقی می‌مانند.

راه‌حل $O(qlogq)$ است.

گفتم: چشمم، گفت: به راهش می‌دار گفتم: جگرم، گفت پر آهش می‌دار

گفتم که: دلم، گفت: چه داری در دل گفتم: غم تو، گفت: نگاهش می‌دار